C# WEB APPLICATION ASSIGNMENT

Smart Service Request Management Portal

1. Assignment Overview

Build a production-style web application called the Smart Service Request Management Portal (SSRMP). The portal will allow employees or customers to submit service requests, attach supporting information, track progress, communicate with support agents, and receive status notifications. Support agents and administrators will use role-based dashboards to assign, prioritize, resolve, and audit requests.

The solution should demonstrate more than basic CRUD operations. It must include secure authentication, authorization, business rules, workflow transitions, asynchronous processing, validation, search, reporting, automated testing, and deployment readiness.

2. Business Scenario

Contoso Support Services currently receives IT, facilities, HR, and general support requests through email and spreadsheets. Requests are sometimes duplicated, assigned to the wrong team, or left without updates. Management cannot reliably measure response times, workload, or service quality.

The organization wants a centralized portal that provides a clear request lifecycle, accountability, and operational reporting. The first release will be used by requesters, support agents, team leads, and administrators.

3. Learning Objectives

- Design a layered and maintainable C# web application.
- Implement authentication and fine-grained role-based authorization.
- Model relational data and manage schema changes with EF Core migrations.
- Enforce business rules and valid workflow transitions.
- Build responsive, accessible, and validated user interfaces.
- Implement logging, error handling, background processing, and notifications.
- Write meaningful unit and integration tests.
- Use source control and produce professional technical documentation.

4. Required Technology Stack

- .NET 8 or later and C#.

- ASP.NET Core MVC or Razor Pages. A Web API plus a separate frontend is acceptable if the API remains in C#.
- Entity Framework Core using Code First Migrations.
- ASP.NET Core Identity or an equivalent secure identity implementation.
- SQL Server or PostgreSQL.
- Bootstrap or another responsive CSS framework.
- xUnit, NUnit, or MSTest for automated testing.
- Git for version control.

5. User Roles

Role	Primary Responsibilities	Access Scope
Requester	Create and track requests; add comments and attachments; confirm resolution.	Own requests only.
Support Agent	Review queue; accept assignments; update status; communicate and resolve requests.	Assigned requests and authorized categories.
Team Lead	Assign/reassign requests; monitor SLA and workload; reopen or escalate requests.	Requests belonging to the lead's team.
Administrator	Manage users, roles, teams, categories, priorities, SLA rules, and system settings.	Full administrative access.

6. Functional Requirements

6.1 Authentication and Account Management

- Allow users to register, sign in, sign out, and reset a forgotten password.
- Require email verification before a requester can submit a service request.
- Lock or throttle authentication after repeated failed sign-in attempts.
- Allow administrators to activate or deactivate accounts and assign roles.
- Deny inactive users access even if their credentials are correct.

6.2 Service Request Creation

- A requester can create a request with title, detailed description, category, priority, preferred contact method, and optional attachments.
- Generate a human-readable reference number such as SR-2026-000123. The number must be unique and generated safely under concurrent requests.
- Validate allowed attachment types and configurable maximum file size. Store attachment metadata in the database and files outside the public web root or in secure object storage.
- Prevent duplicate active requests by warning the user when a similar request with the same category and title was submitted recently. The user may continue after acknowledging the warning.
- Capture created date, creator, current status, and full audit information automatically.

6.3 Request Workflow

Implement the following default lifecycle:

New -> Assigned -> In Progress -> Waiting for Requester -> Resolved -> Closed

The following business rules are mandatory:

- Only team leads or administrators can assign or reassign a request.
- An agent can move an assigned request to In Progress.
- Waiting for Requester requires a public comment explaining what information is needed.
- Resolved requires a resolution summary and resolution category.
- A requester can close a resolved request or reopen it within seven calendar days by giving a reason.
- After seven days, a resolved request is automatically closed by a background job.
- Closed requests cannot be edited except for administrator-only correction notes.
- Every transition must be validated on the server and written to status history.

6.4 Assignment and SLA Management

- Categories are mapped to support teams. New requests should be routed to the correct team automatically.

- Team leads can assign requests manually to agents in their team.
- An administrator can define SLA targets by priority, including response due time and resolution due time.
- Business hours are Monday through Friday, 09:00 to 18:00. Exclude weekends when calculating due dates. Public holiday support is an optional enhancement.
- Visually identify requests that are approaching or have breached an SLA.
- A scheduled process must detect SLA breaches and create an audit entry and notification.

6.5 Comments, Activity, and Notifications

- Users can add comments to a request. Comments can be public or internal; requesters must never see internal comments.
- Comments are immutable. Corrections must be added as new comments.
- Show a chronological activity timeline containing status changes, assignments, comments, attachment events, and automated events.
- Create in-app notifications when a request is assigned, commented on, resolved, reopened, or breaches SLA.
- Email sending may use a local SMTP test server, a provider sandbox, or a mocked service. Notification failures must not roll back the primary business transaction.

6.6 Search, Filtering, and Pagination

- Search by request reference, title, requester name/email, and description keywords.
- Filter by status, category, priority, team, assignee, created date range, SLA state, and ownership.
- Support sorting by reference, creation date, priority, status, and SLA due date.
- Apply server-side pagination. Page size should be configurable.
- Preserve filters while moving between pages or opening and returning from a request.

6.7 Dashboards and Reporting

- Requester dashboard: counts for open, waiting, resolved, and closed requests plus recent activity.

- Agent dashboard: assigned workload, unassigned team queue, requests due soon, and breached requests.
- Team lead dashboard: requests by status and priority, agent workload, SLA compliance, and average resolution duration.
- Administrator dashboard: active users, request volume by category, and configurable reference data.
- Allow authorized users to export a filtered request list to CSV. Export must enforce the same authorization and filters as the screen.

7. Non-Functional Requirements

7.1 Security

- Use authorization policies or handlers rather than relying only on hidden UI controls.
- Protect all state-changing operations against cross-site request forgery.
- Use parameterized data access through EF Core and never concatenate untrusted SQL.
- Apply output encoding and safe file-name handling.
- Do not log passwords, tokens, connection strings, or attachment contents.
- Secrets must be supplied through development secrets or environment variables, not committed to source control.
- Validate object ownership to prevent insecure direct object reference vulnerabilities.

7.2 Reliability and Maintainability

- Use centralized exception handling and show user-friendly error pages.
- Use structured logging with correlation or request identifiers.
- Use asynchronous I/O for database and file operations where appropriate.
- Separate presentation, application/business logic, and persistence concerns.
- Use dependency injection and interfaces for external services.
- Include database seed data for roles, an administrator, categories, teams, priorities, and demo requests.

7.3 Usability and Accessibility

- Pages must work on desktop and mobile widths.

- Forms must provide clear labels, validation summaries, and field-level error messages.
- Use semantic headings, keyboard-accessible controls, sufficient color contrast, and meaningful link text.
- Dangerous operations require confirmation and must clearly state their impact.

8. Minimum Data Model

Entity	Minimum Data to Represent
ApplicationUser	Identity fields, display name, active flag, team membership.
SupportTeam	Name, description, active flag.
Category	Name, mapped team, active flag.
ServiceRequest	Reference, title, description, requester, category, priority, status, assignee, dates, SLA deadlines, resolution data, concurrency token.
RequestComment	Request, author, visibility, body, created date.
RequestAttachment	Request/comment association, original and stored name, content type, size, uploader, created date.
RequestStatusHistory	Request, from status, to status, changed by, reason, changed date.
AssignmentHistory	Request, previous/new assignee, assigned by, reason, date.
SlaPolicy	Priority, response target, resolution target, active flag.
Notification	Recipient, type, message, related request, read flag, created date.
AuditLog	Actor, action, entity, entity identifier, timestamp, safe change summary, correlation identifier.

You may add entities or fields. Explain major modeling decisions, indexes, delete behaviors, and ownership rules in the README or design document.

9. Key Acceptance Criteria

ID	Acceptance Criterion
AC-01	A verified requester can submit a valid request and receives a unique reference number.
AC-02	An unauthorized user cannot view or modify another requester's request by changing the URL.
AC-03	Internal comments never appear in requester screens, exports, or requester API responses.
AC-04	Invalid status transitions are rejected even when submitted directly to the server endpoint.
AC-05	A team lead can assign a team request only to a valid active agent in that team.
AC-06	The system calculates, displays, and records SLA breach state according to the configured policy.
AC-07	The automatic closure process closes eligible resolved requests and records an automated history entry.
AC-08	Search, filters, sorting, and pagination can be combined without exposing unauthorized records.
AC-09	CSV export contains only the currently authorized and filtered rows.
AC-10	Concurrent updates do not silently overwrite newer data; the user receives a clear conflict message.
AC-11	Unexpected errors are logged with a correlation identifier without exposing sensitive details to the user.

AC-12	Automated tests cover the most important business rules and authorization boundaries.

10. Testing Requirements

- At least 15 meaningful unit tests for workflow rules, SLA calculations, routing, validation, or authorization logic.
- At least 5 integration tests using a test server and isolated test database or database container.
- Tests must include positive and negative cases.
- Include at least one authorization test, one concurrency test, and one background-job test.
- Provide brief manual test evidence for responsive layout and file upload behavior.
- The test suite must run with a single documented command.

11. Required Deliverables

- Complete source code in a Git repository with an appropriate .gitignore.
- Database migrations and seed data.
- README containing prerequisites, setup, configuration, database migration, run, test, and demo-user instructions.
- Architecture note describing layers, major design decisions, authentication/authorization approach, and background processing.
- Entity-relationship diagram.
- API or endpoint summary for important operations.
- Automated test project and test results.
- Screenshots showing core workflows and role-based dashboards.
- A short known-limitations section and future improvements.
- Optional: deployment configuration for Azure App Service, Docker, IIS, or another host.

12. Submission Rules

- Commit regularly with clear messages. A single final commit will reduce the source-control score.

- Do not commit secrets, generated build folders, or real personal data.
- All setup steps must work on a clean machine when followed as written.
- Use sample identities and synthetic data only.
- Any third-party package or copied code must be identified in the README with its purpose and license/source.
- The application must compile and the automated tests must run successfully.
- Administrative configuration history with rollback for safe reference-data changes.

16. Suggested Project Structure

```
src/
  ServicePortal.Web
  ServicePortal.Application
  ServicePortal.Domain
  ServicePortal.Infrastructure
tests/
  ServicePortal.UnitTests
  ServicePortal.IntegrationTests
docs/
  architecture.md
  er-diagram.png
```

This structure is a suggestion, not a mandatory architecture. A well-organized single-project solution is acceptable if responsibilities remain clear and the design is justified.

17. Assumptions and Boundaries

- Online payment, live chat, and production email credentials are out of scope.
- The portal supports one organization in the required scope; multi-tenancy is optional.
- Uploaded files do not need preview generation.
- Business-hour SLA calculation is required; public holiday exclusion is optional unless selected as a difficulty enhancer.
- Using mocked email delivery is acceptable, but the design must allow replacement with a real provider.
- The application should use synthetic sample data and must not contain real employee or customer information.

18. Candidate Demonstration Checklist

- Register or sign in as a requester and create a request with an attachment.
- Show automatic team routing and team-lead assignment.
- Update the request as an agent and demonstrate a rejected invalid transition.
- Add an internal comment and prove that the requester cannot see it.
- Resolve, reopen, and close a request according to the workflow rules.
- Demonstrate combined filtering, pagination, and authorized CSV export.
- Show an SLA warning or breach and the resulting history/notification.
- Run automated tests.
- Show logs for a controlled error and explain the correlation identifier.
- Explain one design tradeoff and one future improvement.

19. Final Quality Gate

Before submission, confirm that the solution builds from a clean checkout, database setup is reproducible, migrations apply successfully, seeded demo accounts are documented, tests pass, authorization is enforced on the server, secrets are absent from the repository, and the README matches the actual implementation.